

Capstone Project: Intelligent Credit Card & Rewards Optimization Agent

1. Project Idea

The project is an AI agent that helps users choose the best credit card or reward strategy for a transaction.

A user may ask:

"I am spending ₹50,000 on flights. Which card should I use?"

or

"I have ₹1,20,000 annual hotel spends and ₹80,000 flight spends. Should I use Axis Atlas, HDFC Diners Club Black, or another card?"

The agent does not simply answer from memory. It retrieves card terms, reward rules, exclusions, milestone benefits, caps, and transfer partner information from a vector database, performs calculations using a tool, and gives a grounded recommendation.

The final output should not only say "Use Card X." It should explain:

- Which card gives the highest reward value
- How the calculation was done
- Which rule was used from the card document
- Whether any cap, exclusion, or milestone condition applies
- What assumptions were made
- Whether the user should confirm before taking a complex transfer decision

This makes it a strong capstone because it combines RAG, agents, tool use, memory, guardrails, human approval, evaluation, and monitoring.

2. Why This Is a Strong Capstone Project

This project is useful because credit card reward optimization is a real-world problem where information is scattered across PDFs, bank websites, card terms, reward program rules, and transfer partner charts.

It is also not a simple chatbot problem. The agent must combine three abilities:

First, it must retrieve the correct rule.

For example, the rule for flights may be different from the rule for hotels, wallet reloads, insurance, rent, fuel, education, or international transactions.

Second, it must reason with numbers.

For example, ₹50,000 spent on flights may give 5,000 points on one card, 2,000 points on another card, and a milestone benefit on a third card. The agent should calculate actual value, not just give vague advice.

Third, it must avoid hallucination.

Credit card terms change frequently. The agent should only answer from retrieved documents and should clearly say when it does not have enough information.

This makes the project closer to a real fintech assistant than a basic RAG demo.

3. Target Users

The target users can be:

- Frequent travellers
- Premium credit card users
- People optimizing points for flights and hotels
- Finance content creators
- Card comparison platforms
- Personal finance advisors
- Students learning agentic AI and RAG

The project can start as a learning capstone, but it can later become a serious personal finance tool if updated with current card data and strong compliance guardrails.

4. Example Use Cases

Use Case 1: One-Time Transaction Recommendation

User input:

"I am booking a ₹50,000 domestic flight. I have Axis Atlas, HDFC DCB, and SBI Cashback. Which card should I use?"

Expected agent behavior:

The agent retrieves flight reward rules for all three cards.
It checks whether flights fall under accelerated rewards or normal rewards.

It checks if there are any monthly caps or exclusions.
It calculates expected points or cashback.
It converts the reward into rupee value using a user-defined point valuation.
It recommends the best card.

Example output:

"For this ₹50,000 flight booking, Card A gives the highest estimated value. Based on the retrieved rule, flight bookings earn accelerated rewards. At the assumed point value of ₹1 per point, the estimated reward value is ₹2,500. Card B gives ₹1,500 equivalent value and Card C gives ₹1,000 cashback. So Card A is the best choice, provided your monthly cap has not already been exhausted."

Use Case 2: Monthly Spend Optimization

User input:

"My monthly spends are ₹30,000 on dining, ₹40,000 on travel, ₹20,000 on groceries, and ₹15,000 on utilities. Suggest the best card-wise allocation."

Expected agent behavior:

The agent classifies spend into categories.
It retrieves reward rules by category.
It checks exclusions and caps.
It distributes spends across cards.
It recommends which card to use for each category.

Example output:

"Use Card A for travel because it gives accelerated rewards on flights and hotels. Use Card B for dining if the merchant category is eligible. Use Card C for groceries because it gives direct cashback. Avoid using Card A for utilities because the retrieved terms show utilities are excluded from accelerated rewards."

Use Case 3: Point Transfer Strategy

User input:

"I have 40,000 reward points. Should I transfer them to hotel partners or airline partners?"

Expected agent behavior:

The agent retrieves transfer partner rules.
It asks the user's goal: hotel stays, flights, cashback, or luxury travel.

It calculates different transfer values.
It asks for user approval before giving a final execution route.

Example output:

"You can transfer 40,000 points to Partner A or Partner B. Based on the current retrieved transfer ratio, Partner A may give better hotel value if you redeem during high room-rate periods. Before I calculate a final transfer route, please confirm that you want me to use the currently available partner ratios and assume your target redemption is hotel stay value rather than cashback value."

5. Data Sources

The first version should use 4–5 cards only. This keeps the project manageable.

Suggested card examples:

Axis Atlas
HDFC Diners Club Black
HDFC Infinia
American Express Platinum Travel
SBI Cashback or ICICI Amazon Pay

For each card, collect:

Card terms and conditions PDF
Reward program rules
Category-wise reward structure
Exclusion list
Capping rules
Milestone benefits
Redemption options
Point transfer partners
Point conversion ratios
Annual fee and waiver conditions

Important: do not hardcode all rules directly in the prompt. The project should use documents as the source of truth.

6. Data Modeling Approach

The project should not only store raw PDF chunks. It should store both unstructured and structured data.

A. Unstructured Document Store

This will contain PDF chunks.

Example fields:

document_id
card_name
issuer
document_type
chunk_text
page_number
effective_date
source_url
embedding

This is useful for RAG-based retrieval.

B. Structured Reward Rules Table

This will contain clean reward rules extracted from documents.

Example fields:

card_name
spend_category
reward_type
reward_rate
reward_unit
monthly_cap
annual_cap
exclusion_flag
milestone_condition
source_document_id
confidence_score

This makes calculation easier.

C. Transfer Partner Table

Example fields:

card_name
partner_name
partner_type
transfer_ratio
minimum_transfer_points

maximum_transfer_points
effective_date
source_document_id

D. User Profile Table

Example fields:

user_id
cards_owned
preferred_reward_type
point_valuation
monthly_spend_pattern
preferred_partners
conversation_summary

This enables memory and personalization in later stages.

7. Recommended Tech Stack

Backend

Python
FastAPI
LangChain
LangGraph
PostgreSQL
pgvector
SQLAlchemy

RAG Pipeline

PyMuPDF or pdfplumber for PDF parsing
RecursiveCharacterTextSplitter for chunking
OpenAI / Gemini / local embedding model for embeddings
pgvector for vector similarity search
Hybrid retrieval if possible: keyword + vector search

Agent Workflow

LangGraph for graph orchestration
State memory for multi-turn conversation
Tool calling for retrieval and calculations
Human-in-the-loop node for approval

UI

Streamlit for capstone demo
Optional later: Next.js frontend

Monitoring

LangSmith for traces, latency, token usage, and debugging
Phoenix as an open-source observability alternative
Custom logs in PostgreSQL for user queries and recommendations

Evaluation

RAGAS or custom evaluation scripts
Manual golden test cases
Retrieval precision
Answer faithfulness
Calculation correctness
Hallucination checks

8. System Architecture

The architecture can be explained in five layers.

Layer 1: Data Ingestion

The system ingests credit card PDFs, reward charts, transfer partner rules, and exclusion lists.

Steps:

Upload PDF
Extract text
Clean text
Split into chunks
Add metadata such as card name, issuer, page number, document type, and effective date
Generate embeddings
Store chunks in PostgreSQL with pgvector

Layer 2: Retrieval

When the user asks a question, the system retrieves the most relevant rules.

Example query:

"I am spending ₹50,000 on flights."

The retriever should search for:

flight rewards
travel category
accelerated rewards
excluded merchants
monthly caps
milestone benefits

The retrieved chunks are passed to the LLM along with the user query.

Layer 3: Agent Reasoning

The agent decides what to do next.

It may decide:

Need to ask a follow-up question
Need to retrieve more card rules
Need to call the calculator tool
Need to compare cards
Need human approval before transfer advice
Need to refuse due to insufficient information

Layer 4: Tool Use

The agent should not do all calculations inside the LLM response. It should call a calculator tool.

Example calculator input:

Spend amount: ₹50,000
Reward rate: 5 points per ₹100
Point value: ₹1
Cap: 5,000 points

Example calculator output:

Base points: 2,500
Reward value: ₹2,500
Cap applied: No
Effective return: 5%

Layer 5: Final Recommendation

The final response should be grounded, calculated, and explainable.

It should include:

- Best card
- Estimated reward value
- Effective return percentage
- Rules used
- Assumptions
- Risks or exclusions
- Alternative option
- Confidence level

9. LangGraph Workflow Design

The project should be built as a graph rather than a linear chatbot.

Suggested nodes:

1. User Input Node

Receives user query.

Example:

"I am spending ₹50,000 on flights."

2. Intent Classification Node

Classifies the query into one of these:

- Single transaction recommendation
- Monthly spend optimization
- Point transfer strategy
- Card comparison
- Missing information query

3. Clarification Node

Asks follow-up questions if required.

Example:

"Do you want to optimize for cashback, airline miles, or hotel points?"

4. Retrieval Node

Fetches relevant card rules from vector DB and structured tables.

5. Rule Validation Node

Checks whether retrieved rules are strong enough.

If no reliable rule is found, the agent should not guess.

6. Calculation Node

Calls calculator tool and computes rewards.

7. Comparison Node

Compares all eligible cards.

8. Guardrail Node

Checks:

Did the answer use retrieved evidence?

Did the agent invent a card rule?

Are caps and exclusions mentioned?

Is financial advice framed safely?

Is the confidence level reasonable?

9. Human Approval Node

Used for complex transfer strategies.

Example:

Before suggesting a point transfer route, the UI asks:

"Confirm that you want to calculate transfer routing using currently available partner ratios."

10. Final Answer Node

Generates the final recommendation.

10. Stage-Wise Implementation Plan

Stage 1: Core MVP — RAG + Single Agent

Goal

Build a basic assistant that can answer:

“For this transaction, which card gives the best reward?”

What to Build

PostgreSQL database with pgvector
PDF ingestion pipeline
Chunks from 4–5 card documents
Basic retriever
Single LangChain/LangGraph agent
Simple Streamlit input box or CLI interface

Workflow

User enters:

“I am spending ₹50,000 on flights.”

System retrieves card reward rules.

Agent returns:

Recommended card
Reason
Retrieved rule summary
Basic reward calculation

Example Output

“Based on the retrieved travel reward rules, Card A appears to give the highest reward for a ₹50,000 flight transaction. The estimated reward value is ₹2,500, assuming each point is worth ₹1. Card B gives an estimated ₹1,500 value. Card C gives ₹1,000 cashback. Therefore, Card A is recommended, provided your monthly cap has not been exhausted.”

Stage 1 Deliverables

Working RAG pipeline
Vector database with card documents
Basic card recommendation agent

At least 20 test questions
Basic evaluation sheet comparing expected vs actual answer

Stage 2: Advanced Orchestration, Memory, and Tool Use

Goal

Move from a simple RAG chatbot to a real agentic workflow.

What to Build

LangGraph stateful workflow
Conversation memory
Calculator tool
User preference memory
Structured reward rules table
Multi-turn clarification

New Capabilities

The agent can ask:

“Are you optimizing for cashback, hotel points, or airline miles?”

The agent can remember:

User owns Axis Atlas and HDFC DCB
User values hotel points more than cashback
User usually spends ₹40,000 per month on travel
User does not want cards with complicated redemption

The agent can calculate:

Net reward value
Effective return percentage
Milestone impact
Reward caps
Annual fee recovery

Example Conversation

User:

“I am spending ₹1,00,000 on hotels this month. Which card should I use?”

Agent:

"Do you want to optimize for hotel points or simple cashback?"

User:

"Hotel points."

Agent:

"Based on your preference for hotel points, Card A is stronger if the hotel transaction qualifies under the travel category. The estimated reward value is ₹7,500, including milestone impact. Card B gives ₹4,000 equivalent value. However, if your monthly cap has already been used, Card B becomes better."

Stage 2 Deliverables

Multi-node LangGraph workflow
Memory-enabled conversation
Calculator tool
Card comparison logic
Spend category classifier
Structured reward rule table
More detailed evaluation dataset

Stage 3: Human Approval, Guardrails, and Monitoring

Goal

Make the project production-like.

What to Build

Streamlit UI
Human approval step
Guardrail checks
Trace monitoring
Token and latency logging
Hallucination detection
Feedback collection

Human Approval Example

For simple card recommendations, approval is not needed.

But for complex point transfer strategies, approval is useful.

Example:

User:

"I have 80,000 points. Tell me how to transfer them for maximum hotel value."

Agent:

"I can calculate this, but point transfers are irreversible in many reward programs. Please confirm that you want me to calculate transfer routing using the currently retrieved partner ratios."

The Streamlit UI shows:

Confirm transfer calculation
Cancel

Only after confirmation does the graph move to the transfer calculation node.

Guardrail Examples

The answer must cite retrieved rules.

The answer must not invent reward rates.

The answer must mention caps if found.

The answer must mention exclusions if found.

The answer must say "I do not have enough information" when retrieval is weak.

The answer must not present itself as certified financial advice.

Monitoring

Track:

User query
Retrieved chunks
LLM response
Tool calls
Calculation result
Latency
Token usage
Cost per query
User feedback
Failure reason

Stage 3 Deliverables

Streamlit app
Human-in-the-loop workflow

Guardrail layer
LangSmith or Phoenix tracing
Token and latency dashboard
Final demo video
Final project report

11. Suggested Folder Structure

credit-card-rewards-agent/

app/ streamlit_app.py

backend/ main.py api_routes.py

agents/ graph.py nodes.py state.py prompts.py

tools/ calculator.py retriever.py rule_validator.py transfer_calculator.py

rag/ ingest_pdfs.py chunk_documents.py embed_documents.py retrieval.py

database/ models.py db.py schema.sql

data/ raw_pdfs/ processed_chunks/ sample_queries.csv golden_answers.csv

evaluation/ rag_eval.py calculation_eval.py hallucination_eval.py

monitoring/ langsmith_config.py phoenix_config.py custom_logger.py

tests/ test_calculator.py test_retrieval.py test_agent_outputs.py

README.md requirements.txt .env.example

12. Database Tables

Table 1: card_documents

Stores metadata about uploaded card documents.

Columns:

document_id
card_name
issuer

document_type
effective_date
source_url
uploaded_at

Table 2: document_chunks

Stores RAG chunks.

Columns:

chunk_id
document_id
card_name
chunk_text
page_number
embedding
metadata_json

Table 3: reward_rules

Stores extracted structured reward rules.

Columns:

rule_id
card_name
spend_category
reward_rate
reward_unit
cap_type
cap_value
exclusion_flag
milestone_flag
source_chunk_id
confidence_score

Table 4: transfer_partners

Stores transfer rules.

Columns:

partner_id
card_name
partner_name
partner_type

transfer_ratio
minimum_points
maximum_points
effective_date
source_chunk_id

Table 5: user_profiles

Stores user preferences.

Columns:

user_id
cards_owned
preferred_reward_type
point_valuation
monthly_spend_pattern
preferred_partners

Table 6: recommendation_logs

Stores agent outputs for monitoring.

Columns:

query_id
user_id
query_text
retrieved_chunks
recommended_card
estimated_value
confidence_score
latency_ms
token_usage
created_at

13. Prompting Strategy

The system prompt should force grounded answers.

Example system instruction:

“You are a credit card rewards optimization assistant. You must answer only using retrieved card documents and structured reward rules. If the retrieved evidence is insufficient, say that you do not have enough

information. Do not invent reward rates, card benefits, transfer partners, or exclusions. Always show assumptions, calculation steps, and confidence level.”

The final answer should follow this structure:

Recommended card
Estimated reward value
Calculation
Rules used
Caps or exclusions
Assumptions
Alternative card
Confidence level

14. Example Final Answer Format

User query:

“I am spending ₹50,000 on flights. Which card should I use?”

Agent response:

Recommended card: Card A

Estimated reward value: ₹2,500

Calculation:

Spend amount = ₹50,000
Reward rate = 5 points per ₹100
Estimated points = 2,500
Assumed point value = ₹1
Estimated reward value = ₹2,500
Effective return = 5%

Why this card:

The retrieved card document says that eligible flight bookings earn accelerated rewards. No exclusion was found for this transaction category in the retrieved chunks.

Comparison:

Card A: ₹2,500 estimated value
Card B: ₹1,500 estimated value
Card C: ₹1,000 cashback

Important checks:

This assumes the transaction is made with an eligible merchant.
This assumes your monthly reward cap is not already exhausted.
This assumes each reward point is worth ₹1.
If your point valuation is lower, the recommendation may change.

Confidence: Medium-High

Reason for confidence:

The recommendation is based on retrieved reward rules and calculator output. Confidence is not marked High because monthly cap usage is unknown.

15. Evaluation Plan

The project should be evaluated on three levels.

A. Retrieval Evaluation

Check whether the retriever fetches the correct rule.

Metrics:

Precision@K
Recall@K
MRR
Context relevance

Example test:

Query:

“₹50,000 flight spend on Axis Atlas”

Expected retrieved chunk:

Flight or travel reward rule for Axis Atlas

B. Calculation Evaluation

Check whether the calculator gives correct numbers.

Example test:

Spend = ₹50,000
Reward rate = 5 points per ₹100
Point value = ₹1
Expected value = ₹2,500

The output should exactly match the expected answer.

C. Answer Quality Evaluation

Check whether the final response is useful and safe.

Criteria:

Does it recommend the right card?
Does it explain the calculation?
Does it mention assumptions?
Does it mention caps or exclusions?
Does it avoid hallucinated benefits?
Does it say "not enough information" when retrieval is weak?

16. Sample Test Cases

Test Case 1

Input:

"I am spending ₹50,000 on flights."

Expected behavior:

Retrieve travel reward rules.
Compare cards.
Calculate reward value.
Recommend best card.

Test Case 2

Input:

"I am paying ₹25,000 insurance premium. Which card should I use?"

Expected behavior:

Check whether insurance is eligible or excluded.
Avoid assuming regular reward rate.
Mention exclusion if found.

Test Case 3

Input:

"I am spending ₹1,00,000 on hotels and want Marriott points."

Expected behavior:

Ask or use preference for hotel points.
Retrieve hotel/travel reward rules.
Retrieve transfer partner rules.
Calculate value.
Ask approval if transfer routing is involved.

Test Case 4

Input:

"Which card is best for rent payment?"

Expected behavior:

Retrieve rent category rules.
Check exclusion/capping.
If no rule is available, say information is insufficient.

Test Case 5

Input:

"I have 50,000 points. Should I convert them to cashback?"

Expected behavior:

Ask the user's goal.
Compare cashback value vs travel or hotel transfer value.
Mention assumptions.

17. Guardrails

This project needs strong guardrails because financial information can be sensitive and card terms change frequently.

Guardrails should include:

Do not answer without retrieved evidence.

Do not invent reward rates.

Do not invent transfer partners.

Do not ignore exclusions.

Do not ignore caps.

Do not present the answer as official financial advice.

Mention effective date if available.

Mention that users should verify with the issuer before making irreversible transfers.

Ask for approval before complex transfer strategy.

18. Capstone Demo Flow

The final demo can be shown like this:

First, show the uploaded card PDFs.

Second, show the ingestion pipeline:

PDF → text extraction → chunks → embeddings → pgvector.

Third, ask a simple query:

"I am spending ₹50,000 on flights."

Show retrieved chunks and final recommendation.

Fourth, ask a multi-turn query:

"I spend ₹1,00,000 per month on travel. Optimize my cards."

Show how the agent asks follow-up questions.

Fifth, ask a complex transfer query:

"I have 80,000 points. Where should I transfer them?"

Show the human approval button.

Sixth, show monitoring:

Trace
Tool calls
Retrieved chunks
Latency
Token usage
Final response

This demo clearly shows RAG, agents, tool use, memory, guardrails, and observability.

19. Final Capstone Deliverables

The final submission should include:

Working Streamlit app
PostgreSQL vector database
PDF ingestion pipeline
LangGraph agent workflow
Calculator tool
Card recommendation logic
Human approval flow
Guardrail checks
Monitoring dashboard
Evaluation dataset
Final project report
Demo video
GitHub repository with README

20. Why This Project Stands Out

Most RAG projects only retrieve information and summarize it.

This project goes further.

It retrieves financial rules, validates them, calculates reward value, compares alternatives, remembers user preferences, asks clarification questions, prevents hallucination, and uses human approval for risky decisions.

That is why this is a strong capstone project. It demonstrates that the student understands not just RAG, but how to build a real agentic AI system around a practical use case.

The best version of this project is not a chatbot that says “Axis Atlas is good for travel.” The best version is an explainable decision engine that says:

"For your ₹50,000 flight booking, Card A gives the highest value based on the retrieved travel reward rule. The estimated return is 5%, assuming the merchant is eligible and your cap is unused. Card B gives 3%, and Card C gives 2%. If your monthly cap on Card A is already exhausted, use Card B instead."

That is the difference between a basic GenAI demo and a serious capstone project.